

CS-1004 Object Oriented Programming (CS)

Final Exam

Date: May 20, 2024

Course Instructor(s):

Dr. Ali Zeeshan, Mr. Shehryar Rashid, Ms.
Mariam Hida

Total Time (Hrs): 3
Total Marks: 150
Total Questions: 5

Roll No

Course Section

Student Signature

DO NOT OPEN THE QUESTION BOOK OR START UNTIL INSTRUCTED.

Instructions:

1. Read the questions carefully, understand the question, and then attempt it on answer sheet.
2. Any answer provided on question paper will not be considered for marking. Therefore, avoid writing anything on question paper as final answer.
3. After asked to commence the exam, please verify that you have eight (8) different printed pages including this title page. There are a total of five (05) questions.
4. Calculator sharing is strictly prohibited.
5. Use permanent ink pens only. Any part done using soft pencil will not be marked and cannot be claimed for rechecking.
6. For additional 5 marks, please address all parts of a question in a single response. And mention the question and part no for each solution clearly on top of each answer. Write your roll number clearly on both answer sheet and question paper.
7. Use a permanent pen to clearly cross out any rough work.

Do not write below this line.

Attempt all the questions.

Q1. What would be the output produced by executing the following C++ code? Identify errors, if any (Either write output or error, both will not be acceptable). Please note that `#include<iostream>` and using namespace std; already included in code. [10+20+5+10=45 Marks]

```
a. class H {  
    public:  
        H() { cout << " H-C "; }  
        ~H() {  
            cout << " H-D " << endl;  
        }  
};  
  
class E {  
    public:  
        E() { cout << " E-C "; }  
        virtual ~E() { cout << " E-D "; }
```

```
};
class F : public E {
public:
    ~F() { cout << " F-D "; }

};
class A {
private:
    H h;
    E* ep;
public:
    A(E* t) : ep(t) { cout << " A-C "; }
    virtual ~A() {
        cout << " A-D ";
        delete ep;
    }
};
class B : public A, public F {
private:
    H h2;
public:
    B(E* t) : F(), A(t) { cout << " B-C "; }
};
int main() {
    B b(new E);
    cout << endl << " ----- " << endl;
    B b2(new F);
    cout << endl << " ----- " << endl;
    return 0;
}
```

b. Please note that there are no syntax errors in this program

```
template<typename T, typename S>
class Matrice {
public:
    Matrice(int r, int c) :
        rows(r), columns(c), ptr(new T[r * c]) {}
    ~Matrice() {
        cout << *this;
        delete[] ptr;
    }

    void operator=(S* lmat) {
        for (int i = 0; i < rows * columns; ++i){
            ptr[i] = lmat[i];
        }
    }
    void operator ++(int dummy){
        for (int i = 0; i < rows * columns; i++)
            ptr[i]++;
    }
    T& operator()(int i, int j) {
        delete &(++*this);
        return ptr[j + i * columns];
    }

    T& operator[](int i) { return ptr[i]; }
```

```
Matrice<T,S>& operator++() {
    Matrice<T,S> *M=new Matrice(columns,rows);
    *M=(this->ptr);
    for (int i = 0; i < rows * columns; i++)
        ptr[i]++;
    return *M;
}
int GetRows() { return rows; }
int GetCols() { return columns; }
private:
    int rows, columns; T* ptr;
};

template<typename T, typename S>
ostream& operator <<(ostream& cout, Matrice<T, S>& m) {
    cout << " Matrix = " << endl;
    for (int i = 0; i < m.GetRows() * m.GetCols(); ++i)
    {
        cout << m[i] << " ";
        if ((i + 1) % m.GetCols() == 0)
            cout << endl;
    }
    return cout;
}

int main() {
    int a[] = { 10, 30, 20, 40, 50, 55, 70, 80 };
    Matrice<int, int> v(4, 2);
    v = a;
    int z = ++v(2, 1);
    cout << " Z= " << z << endl;
    return 0;
}
```

c.

```
struct node {
    int d;
    node* ptr;
};

class list {
    node* here;
public:
    list(node* p = NULL) : here(p) {}

    node* gethere() { return here; }

    void sethere(node* p) { here = p; }

    node* introduce(int x, node* here)
    {
        if (here == NULL)
        {
            here = new node;
            here->d = x;
            here->ptr = NULL;
            return here;
        }
    }
}
```



```

    }
    else
    {
        here->ptr = introduce(x, here->ptr);
    }
    return here;
}

void show(node* here)
{
    if (here == NULL)
    {
        cout << endl;
        return;
    }
    else
    {
        cout << here->d << " ";
        show(here->ptr);
    }
}

node* wonder(node* here)
{
    if (here == NULL || here->ptr == NULL)
        return here;
    node* here2 = wonder(here->ptr);
    here->ptr->ptr = here;
    here->ptr = NULL;
    return here2;
}

};

int main()
{
    list d;
    d.sethere(d.introduce(73, d.gethere()));
    d.sethere(d.introduce(66, d.gethere()));
    d.sethere(d.introduce(91, d.gethere()));
    d.sethere(d.introduce(21, d.gethere()));
    d.sethere(d.introduce(13, d.gethere()));
    d.show(d.gethere());
    d.sethere(d.wonder(d.gethere()));
    d.show(d.gethere());
}

```

d. int x = 5;

```

class Carl {
    int x;

public:
    Carl(int x = 2) : x(x + 2) { cout << "Carl " << x << endl; }
    ~Carl() { cout << "~Carl " << this->x << endl; }
    int& getX() { return this->x; }
};

class Car2 {
    Carl* a2, a1;
    int x;
}

```

```

public:
    Car2() : x(1), a1(--::x), a2(&a1) { cout << "Car2 " << x << endl; }
    Car2(int x) : a2(new Car1(3)), x(x), a1(x++) { cout << "Car2 " << x << endl; }
    ~Car2() { cout << "~Car2 " << Car2::x << endl; }
};

class Car3 {
    Car1& a;
    int x;

public:
    Car3(Car1& a) : x(a.getX()++), a(a) { cout << "Car3" << endl; }
    ~Car3() { cout << "~Car3 " << a.getX() << " " << x << endl; }
};

class Car4 {
    Car3* c;
    Car2* b;

public:
    Car4(Car3* c) : c(c), b(new Car2) { cout << "Car4" << endl; }
    Car4(Car2* b, Car1* a) : b(new Car2(*b)), c(new Car3(*a)) { cout << "Car4" << endl; }
    ~Car4() { cout << "~Car4" << endl; delete b; }
};

int main() {
    Car2 b(1);
    Car1 a;
    Car4 d1(&b, &a);
    cout << "-----" << endl;
    return 0;
}

```

Q2. For the given case study, generate a class diagram representing all classes of the system and what relationship exists between them. For each class, identify and include member variables and member functions. Use (+) to show public components, (-) for private component and (#) for protected components of the class. [20 Marks]

Blood donation management system is a web-based system that enables individuals who want to donate blood to help the needy. It also enables hospitals to record and store the data for people who want to communicate with them, and it also provides a centralized blood bank database. The system targets three types of users: the public who wants to donate blood, the recipients who need the donated blood, and the hospitals who work as an intermediary to manage the communication between the donors and recipients.

The main objective for developing the system is to educate the community on the benefits of blood donation, develop a web-based blood bank system to manage the records of donors and recipients, and encourage voluntary blood donation, easily accessing any information about blood type and the distribution of the blood in various hospitals, based on the hospital needs. This system provides various functionalities. Admin can observe and maintain the whole system namely from maintain donor details, update, and change database. Donors who can donate voluntarily have access to an individual account. Recipients view the system and search for a particular blood requirement and types in need. It facilitates donor registration and donor management by recording their physical and medical statistics. The

system has a facility to Search a blood donor via the internet by using social media like Twitter and Facebook. It also manages the list of donors who are eligible for donation on a particular date with contact number.

The inventory management in blood bank is responsible for storage and issuance of blood. It keeps records of blood requisition and issuance of blood. The salient feature is online request transferring of blood from one blood bank to another. By using this system, donors can view their donation records, including where and when they made donations, and the blood results for each, to learn of their donated blood quality and schedule their next donations. The system uses a First-In-First-Out stock management, where the blood stock that is checked-in to the system first will be the first one given to the hospital when requested. When the blood stock is expired, the system prompts for removing the stock from the inventory and updating the system.

The hospital can request blood via using the online blood bank system. This website will enable the public to make online reservations and includes online advertising for all the blood donation events. It will generate alerts to get all the details about the donation camps arranged by different groups. The system enables admin to generate a report to summarize all records including blood donation, blood requests and blood stock.

Q3. Analyze the following problem & write C++ recursive function.

[20 Marks]

Constraints:

You are not allowed to use loops or static/global variables in this problem. Helping function must also be recursive. 4 marks are reserved for function prototype and main function.

Flood fill:

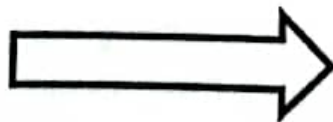
An image is represented by an integer grid image (row X col) where image[i][j] represents the pixel value of the image.

You are also given six integers row (row size of image), col (col size of image), sr (row index), sc (col index), oldColor (previous color) and newColor (new color to be filled). You should perform a flood fill on the image starting from the pixel image[sr][sc].

To perform a flood fill, consider the starting pixel, plus any pixels connected 4-directionally to the starting pixel of the same color as the starting pixel, plus any pixels connected 4-directionally to those pixels (also with the same color), and so on. Replace the color of all of the aforementioned pixels with color.

Example:

1	1	1
1	1	0
1	0	1



2	2	2
2	2	0
2	0	1

Input: image = [[1,1,1],[1,1,0],[1,0,1]], row = 3, col = 3, sr = 1, sc = 1, oldColor = 1, newColor = 2

Output: [[2,2,2],[2,2,0],[2,0,1]]

Explanation: From the center of the image with position [sr, sc] = [1, 1] (e.g. 1 means red pixel), all pixels connected by a path of the same color as the starting pixel are colored with the new color (e.g. 2 means blue pixels).

Note the bottom corner is not colored 2, because it is not 4-directionally connected to the starting pixel.

Q4: Your goal in this question is to design a mobile application management system in other words an "App Store". In the application store you can store information about different type of applications such as system and entertainment applications. For all the types of applications you will store application id (integer), programmer id (integer), and date of publishing. System applications include number of threads they can use and number of hardware devices they can access. Entertainment applications have predefined storage limit and already defined features. Apart from the applications information, "App Store" also stores user information, i.e. his name and email. Furthermore, it also records for each user what applications he is currently using. [5+10=15 Marks]

- Now given the description, identify all the classes, their data members and their relationships (composition, aggregation, and inheritance) for the complete system.
- Implement the identified classes without their member functions. Main is not required.

Q5: Case Study: Academic Institution System.

[15+20+5=40 Marks]

You are tasked with designing a system for an academic institution that consists of different types of employees, including administrators, faculty members, and staff.

You have 6 classes in total, as listed below:

Employee ✓	Administrator ✓	Faculty ✓
Staff	Professor ✓	TeachingAssistant ✓

Generally in an academic institution, all employees have a name, an ID number associated with them and different salaries, depending on their designation. However, certain job have additional information specific to their work that is required to maintain by the system.

An administrator administers a department, as they are responsible for day to day working of that specific department. Faculty is categorized on the basis of department as well, for example computer science faculty would come under the computer science department. While a Professor is a member of the faculty, they have a certain research area, in which they conduct their research over the years. Finally, in this system, the teaching assistant is given the primary task of teaching a course.

With the above description, you are required to design, develop and code the different classes, keeping in view each class' requirements. Please note that there cannot be any redundancy when it comes to attributes (multiple classes cannot have common attributes). Finally, while designing and coding your classes, ensure the following:

- Attributes of all the classes have **private** access modifiers. You are not allowed to create any new attributes, except for those mentioned above.
 - No getter/setter functions are allowed.
 - Every class must have a **display()** function that displays the **complete** information available.
 - String class is not allowed.** Using strings in your code will result in severe penalization of marks.
 - Ensure that you **only provide methods that are necessary** for the proper functioning of the classes.
 - Default constructor (i.e. a constructor with no parameters) is not allowed.** You must have a parameterized constructor.
 - Ensure that you handle **memory allocation/deallocation properly**.
 - You are encouraged to use C++'s **strcpy()** function, as described below:


```
char* strcpy( char* dest, const char* src );
```
- Draw a UML diagram representing the class hierarchy for the Academic Institution System. Illustrate the relationship between the aforementioned classes. Ensure that you have included the necessary attributes and methods in your UML diagram. Generate a class diagram representing all classes of the system and what relationship exists between them. For each class, identify and include member variables and member functions. Use (+) to show public components, (-) for private component and (#) for protected components of the class.
 - Write C++ code for the classes described in the UML diagram.
 - Demonstrate polymorphic behavior by creating objects of different employee types and calling their display functions in the main() function. Create a single array of objects, store the information as listed in the table below and display it using a for-loop.

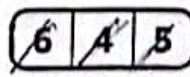
Object Type	Name	ID	Salary	Department	Additional Information
Professor	John Doe	P001	80000	CS	AI (research area)
TeachingAssistant	Jane Smith	TA001	30000	Mathematics	Calculus (course taught)
Adminstrator	Jack Black	AD001	60000	Administration	-

Bonus Question [5 Marks]

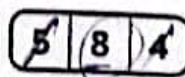
Write the correct answer on the first page of your answer sheet. Clearly mention "Bonus" on top of your answer.



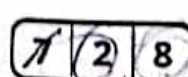
Crack the code.



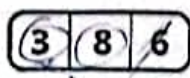
Nothing is correct



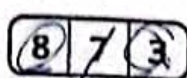
One number is correct, but wrongly placed



Two numbers are correct. Both wrongly placed



Two numbers are correct, but wrongly placed



Two numbers are correct. One is well placed, and the other wrongly placed

